llm-smart-router

A CLI for routing and orchestrating multi-model AI work

Project 2 Report

June Gu

06-642 · Spring 2026

Carnegie Mellon University

1. The Idea

For this project, I built `llm-smart-router`, a standalone TypeScript CLI that helps a user send a task to a suitable AI model and save the result for later review.

The problem I wanted to solve is simple: different models are good at different kinds of work, but most workflows still treat them as one generic chat window. I wanted one command-line tool that could choose a good model for smaller requests and manage a small team workflow for larger ones.

The main user is a developer or technical user who already works across multiple AI systems and wants one CLI for choosing a model, running a team workflow, and reopening previous results.

A typical team command looks like this:

```
smart-router team run --no-interactive "Compare Python vs Rust for building a web scraper. I need: 1) a technical comparison of both languages for web scraping, 2) a practical code example in each language"
```

That command asks the tool to plan the work, assign teammate roles, run those steps, and combine the results into one response. In the final evidence for this report, I reopen a saved successful session instead of showing a fresh successful run because the local provider credentials and quotas were not stable during the final capture pass.

What Is Different From Project-1

My project-1 was `plateprep`, a Python CLI for validating, merging, and summarizing 96-well absorbance plate-reader CSV exports. That project solved one real lab preprocessing problem very well, but it was deliberately narrow in scope.

`llm-smart-router` is a new project rather than a small extension of `plateprep`.

Category	Project-1: `plateprep`	Project-2: `llm-smart-router`
Primary problem	Lab CSV preprocessing	AI workflow orchestration
Audience	Lab users handling plate-reader exports	Developers and technical users working across LLM providers
Interface	Simple Python CLI with three subcommands	TypeScript CLI with routing, team mode, presets, and session review
Input type	Structured CSV files	Natural-language prompts, git diffs, and selected files
Output type	Clean CSV tables and statistics	Routed LLM responses, saved team sessions, and synthesized reviews
How much work it covers	One narrow lab task	A broader developer workflow tool
What had to be built	Parser, validation logic, and CLI	Router, model catalog, provider integrations, team workflows, saved sessions, and tests

The main improvement I define here is that `llm-smart-router` handles a wider range of tasks and supports multi-step teamwork. `plateprep` automated one repeated data-cleaning step. `llm-smart-router` aims to automate a broader class of technical work.

2. The Plan

Before implementing anything, I planned this as a standalone Node/TypeScript project with one CLI name, `smart-router`, and two main workflow layers:

- `single-query routing` — classify a task and pick a best-fit model
- `team mode` — plan specialist roles, run teammates, and synthesize the result

Planned command surface:

- `smart-router "<message>"`
- `smart-router team run ...`
- `smart-router team presets`
- `smart-router team sessions`

- `smart-router team review <id>`
- `smart-router models`
- `smart-router config ...`

Planned Inputs & Outputs

Inputs: a natural-language task or prompt, optional git diff context, optional file context, and configured provider credentials or CLI subscriptions.

Outputs: a routed single-model response, a planned multi-teammate run with status and cost reporting, a synthesized final report from team mode, and saved team sessions for later review.

Planned Libraries

- Node.js + TypeScript
- `commander` for the CLI
- HTTP clients for provider APIs
- `vitest` for automated tests
- Next.js for an optional local web UI

Planned Edge Cases

- No API keys configured
- Provider authentication failures
- Request timeouts or quota failures
- Team-member failure during a multi-step run
- Requests that should still produce a useful synthesis even when one or more teammates fail

3. How You Built It

Initial Prompt

I first used Claude Code to turn the idea into a concrete CLI design before asking it to implement anything. My core implementation prompt was essentially:

> *Build a standalone TypeScript CLI called `llm-smart-router` that routes a prompt to the best available LLM and can run multi-model teammate workflows from the terminal.*

Key Follow-Up Prompts

After that, I guided Claude Code toward a specific shape:

- Keep the project standalone and installable as its own repository
- Make the CLI the primary interface, with the web UI as optional
- Support both single-query routing and a team mode
- Add preset templates for common tasks like code review and debugging
- Save team sessions so previous runs can be revisited
- Add automated tests for router logic, provider behavior, team orchestration, sessions, and git-context support

Where I Corrected the AI

Two corrections mattered most.

First, I pushed the implementation toward explicit request timeouts. An earlier version could hang indefinitely when a provider did not respond. I wanted the CLI to fail clearly instead of silently waiting forever.

Second, I pushed the team mode toward better fallback behavior. In early versions, if one teammate's chosen model failed, that delegated task simply stopped. I pushed the implementation toward trying another model in the same category before giving up.

More broadly, I also steered the project story away from "just another API wrapper" and toward a clearer value proposition: one CLI that can route work, coordinate teammates, and preserve prior sessions for review.

4. Evidence It Works

All commands below were run from the standalone repository at `/Users/junemog/Documents/GitHub/llm-smart-router`. Full transcripts are saved in `evidence/`.

Install

Standard local setup:

```
npm install
npm run build:cli
npm link
```

The captured transcript shows a clean dependency install, a successful CLI build, and a working editable global link for `smart-router`.

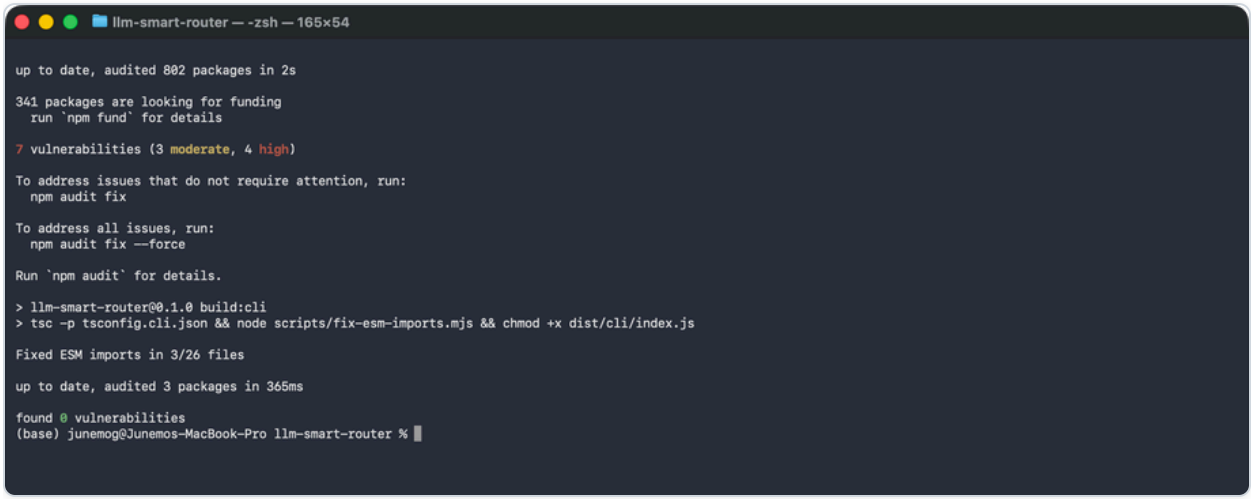

Figure 1. Install, build, and global link flow for the standalone CLI.

Help Output

COMMAND

```
smart-router --help
```

OBSERVED OUTPUT (EXCERPT)

```
Usage: smart-router [options] [command] [message...]
...
Commands:
  config
  team [message...]
  models
  serve [options]
```

This confirms that the CLI surface is exposed correctly and that the primary subcommands are discoverable from the terminal.

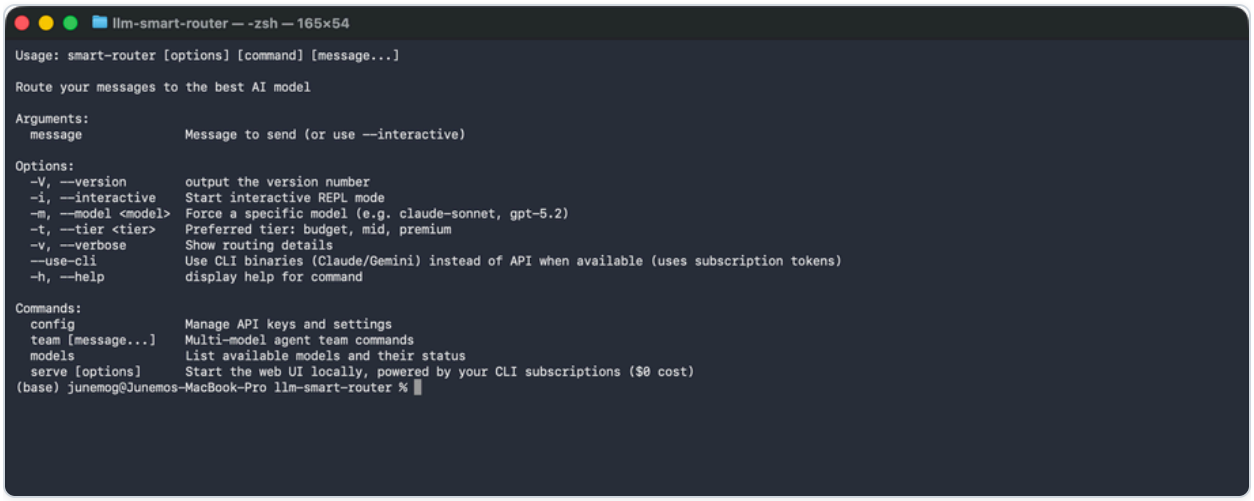

Figure 2. Full `--help` output showing the main command surface.

Model Catalog

COMMAND

```
smart-router models
```

OBSERVED OUTPUT (EXCERPT)

```
Available Models:
  ✓ GPT-4.1 Nano
  ✓ DeepSeek V3.2
  ✓ Gemini 2.5 Flash
  ✓ GPT-5.2
  ✓ Claude Sonnet 4.6
```

This shows that the tool can enumerate the model catalog and distinguish which providers are currently available in the local environment.

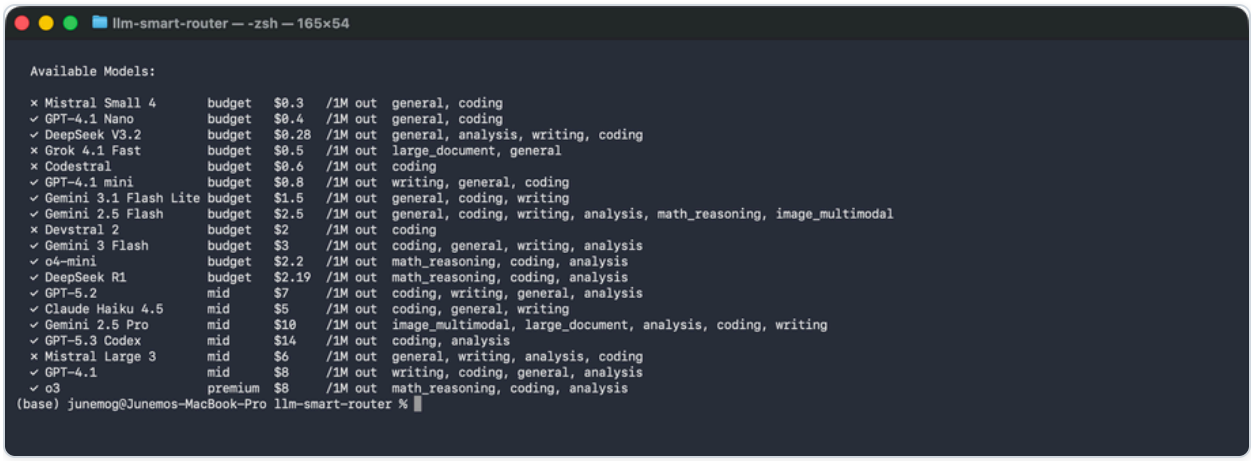

Figure 3. Model catalog output with provider availability and pricing tiers.

Team Presets

COMMAND

```
smart-router team presets
```

OBSERVED OUTPUT (EXCERPT)

```
Available Team Presets:
  code-review   Code Review Team
  debug         Debug Team
  explain       Code Explainer Team
  refactor      Refactoring Team
```

This shows that multi-step workflows are already packaged into reusable preset teams rather than requiring the user to manually define specialist roles every time.

Figure 4. Built-in team presets for review, debugging, explanation, and refactoring.

Example Run 1: Listing Saved Team Sessions

COMMAND

```
smart-router team sessions
```

OBSERVED OUTPUT (EXCERPT)

```
Saved Team Sessions:
[1] Compare Python vs Rust for building a web scraper...
[4] Compare the pros and cons of Python vs Rust for building a web scraper...
Use: smart-router team review <number> to view details
```

This shows that team-mode outputs persist as named artifacts that can be listed later from the CLI. The session list uses numbers, but the review command also accepts a filename or a partial session ID. In this report I use the partial ID `22afb8d1` in the next step because it is more stable than a changing list number.

Figure 5. Saved team sessions listed from the CLI.

Example Run 2: Reviewing a Saved Successful Session

COMMAND

```
smart-router team review 22afb8d1
```

OBSERVED OUTPUT (EXCERPT)

```
Team Session Review
Request: Compare the pros and cons of Python vs Rust for building a web scraper...
Status: done
Cost: $0.093648
Time: 97.9s

Teammates:
✓ Analyst
✓ Coder
```

This is the strongest successful end-to-end artifact in the repository. It shows a completed saved team run with the original request, completion status, runtime, cost, and the two teammate roles. The command also prints the saved synthesis below that metadata, and it can reopen the session by partial ID.

```
llm-smart-router — -zsh — 165×54

  Team Session Review
  ═══════════════════════════════════════
  Request: Compare the pros and cons of Python vs Rust for building a web scraper. I need: 1) A technical comparison of both languages for web scraping, 2) A practic
al code example in each language
  Status: done
  Cost: $0.093648
  Time: 97.9s

  Teammates:
  ✓ Analyst         Claude Sonnet 4       $0.031212
  ✓ Coder           Claude Sonnet 4       $0.062310

  ═══════════════════════════════════════
  Synthesis:
  ═══════════════════════════════════════

# Web Scraping: Python vs Rust Comparison Report

## 1. Brief Summary

Python excels in development speed, ecosystem maturity, and ease of learning, making it ideal for rapid prototyping and data-heavy scraping tasks. Rust offers superi
or runtime performance, memory efficiency, and deployment simplicity, making it better suited for high-volume, production-grade scraping systems. The choice depends
primarily on project requirements around performance needs versus development velocity.

## 2. Key Findings from Each Teammate

### Analyst's Technical Comparison Findings:

    * **Performance**: Rust demonstrates 2-5x faster execution and 2-4x lower memory usage than Python
    * **Ecosystem**: Python has more mature, extensive libraries (BeautifulSoup, Scrapy, requests) while Rust's ecosystem is growing but less comprehensive
    * **Development Experience**: Python enables 2-3x faster initial development with gentler learning curve; Rust provides compile-time safety guarantees
    * **Use Case Recommendations**:
        * Choose Python for: rapid prototyping, data analysis integration, moderate-volume scraping, teams with Python expertise
        * Choose Rust for: high-volume scraping, production systems, resource-constrained environments, long-running scrapers
    * **Maintenance & Deployment**: Rust offers single-binary deployment with minimal dependencies; Python has easier containerization but potential dependency confl
icts

### Coder's Practical Implementation Findings:

    * **Python Code Characteristics**:
        * Demonstrates both synchronous (`requests` + `BeautifulSoup`) and asynchronous (`aiohttp`) approaches
        * Emphasizes practical concerns: error handling, rate limiting, user-agent headers
        * Shows concurrent scraping patterns with configurable concurrency limits
(base) junemog@Junemos-MacBook-Pro llm-smart-router %
```

Figure 6. Saved successful team review reopened by partial session ID.

Automated Tests

The project includes 12 test files with 154 passing tests. Running `npm test` from the project root produces `154 passed (154)`. The full transcript is saved in `evidence/10_tests.txt`.

I also ran `npm run lint` after restructuring the standalone repository. It completed cleanly, with the full transcript saved in `evidence/11_lint.txt`.

Error Handling in the Current Environment

COMMAND

```
HOME=/tmp/llm-smart-router-empty smart-router "hello"
```

OBSERVED OUTPUT

```
No API keys configured.
Run: smart-router config set <provider> <key>
Example: smart-router config set deepseek sk-xxx
```

This confirms that the CLI has a clean failure path for an unconfigured environment and gives the user a direct recovery command instead of a generic stack trace.

I was not able to capture a fresh successful provider-backed run in the current environment because the local provider credentials and quotas were not in a clean state. For that reason, the main success evidence in this report is the previously completed saved session above. The current-environment example here is included only to show the error path for an unconfigured system.

Figure 7. Error handling when no provider credentials are configured.

5. Reflection

What Worked Well When Collaborating With Claude Code?

Claude Code was most helpful in turning a broad product idea into a concrete CLI quickly. It was effective at helping scaffold the routing logic, provider integrations, team workflow, and tests once the direction was clear.

What Did Not Work, and Where Did the AI Fall Short?

The AI was weakest when reliability details mattered. It was initially too optimistic about provider behavior and did not handle timeouts, stale credentials, and teammate failure strongly enough without explicit direction.

How Did You Troubleshoot Problems?

I kept moving from broad product prompts toward environment checks and direct verification:

- Checking `smart-router --help` and `smart-router models`
- Attempting fresh routed queries and team runs
- Reviewing saved session artifacts when provider issues blocked a clean fresh run
- Using `npm test` and `npm run lint` after packaging changes

That pattern made it easier to catch the difference between "the design seems reasonable" and "the CLI actually behaves correctly in a real terminal."

Would You Actually Use This Tool? What Would It Take To Make It Production-Ready?

Yes. I would actually use `llm-smart-router` because I already switch between providers and often want a better way to route work than manually choosing one model at a time.

To make it more production-ready, I would add:

- Stronger provider health checks before a run starts
- Clearer fallback and retry reporting for failed teammates
- Clearer setup documentation for API mode versus CLI-backed mode
- Broader end-to-end tests against mocked provider failures